

1. Dataset Relevance Prompt

Function: `llm_select_best_model_by_dataset`

This prompt is used to determine which benchmarking metrics (columns) are most relevant to a specific user task.

"You are an expert in LLM benchmarking.

We have the following dataset metrics available in columns:

{dataset_cols}

Task description: {description}

Which column(s) are most relevant for evaluating models on this task?

Return the column name(s) as a Python list of strings, e.g. ["MMLU-Pro"]."

2. Model Selection Prompt

Function: `llm_select_best_model_by_dataset`

Once the relevant dataset is identified, this prompt asks the LLM to choose the best-performing model from a filtered top-5 list.

"You are an expert evaluating LLM performance.

Here are the top 5 models for dataset '{best_col}':

{model_summary}

Evaluate them based on:

- Overall reasoning quality
- Reliability
- Practical applicability across common use cases
- Price to performance ratio

Task description: {description}

Return ONLY the best model name. No explanation."

3. Tool Determination Prompts

Function: `llm_determine_tools`

These prompts vary slightly depending on which LLM provider is active, but all aim to extract a JSON list of required tools.

OpenAI Variation:

"You are an expert AI agent builder.

Task description: {description}

Determine which tools this agent should use.

Respond in pure JSON format with a key 'tools', listing tool names.

Example output:

```
{
```

```
"tools": [
```

```
"SearchTool",
```

```
"FileIO",
```

```
"WebBrowser",
```

```
"PythonExecutor"
```

```
]
```

```
}"
```

4. Feature Standardization Prompt

Function: `llm_generate_feature_values`

This prompt classifies the task into a rigid schema used for rule evaluation (e.g., determining memory types).

"You are an expert in AI system classification.

Task description: "{description}"

Strictly assign standardized feature values based on the categories below.

Do NOT invent new values. Use ONLY the exact words listed.

Output JSON object with these keys:

```
{list(allowed_values.keys())}
```

For each key, select ONE value from its allowed list.

Example output:

```
{
```

"Retrieval_Standardized": "RAG",

"Persistence_Standardized": "STM",

"Knowledge_Standardized": "Internal",

...

}

Allowed categories:

{json.dumps(allowed_values, indent=2)}